PEER LEARNING

Explication détaillée du code

Rôle de chaque balise, attribut et règle CSS du projet

Projet universitaire L1 — Site statique HTML5 / CSS3 (sans JavaScript)

Introduction

Ce document explique, fichier par fichier, le rôle de chaque balise HTML, attribut et règle CSS utilisés dans le site Peer Learning. Le site est entièrement statique : HTML5 et CSS3 uniquement, sans JavaScript, sans backend et sans dépendance autre que Bootstrap 5 (grille CSS uniquement) et Font Awesome (icônes).

Toutes les pages partagent la même structure d'en-tête (<head>), la même barre de navigation et le même pied de page. Ces parties communes sont expliquées une seule fois, dans le chapitre consacré à index.html, puis seules les différences sont signalées pour les pages suivantes.

Fichiers couverts

Fichier	Rôle
`index.html`	Page d'accueil : bandeau hero, statistiques, 3 cours populaires
`courses.html`	Catalogue complet des 6 cours (cartes)
`course-1.html à course-6.html`	Une page de détail statique par cours
`tutors.html`	Liste des 6 tuteurs
`resources.html`	Liste des ressources téléchargeables (simulation)
`contact.html`	Formulaire de contact avec validation HTML native
`css/style.css`	Feuille de style unique pour toutes les pages

1. index.html — Page d'accueil

1.1 En-tête du document (<head>)

La section <head> ne s'affiche pas à l'écran : elle contient les métadonnées et les liens vers les feuilles de style.

Balise / attribut	Rôle
`<!DOCTYPE html>`	Indique au navigateur qu'il s'agit de HTML5. Toujours la première ligne du fichier.
`<html lang="fr">`	Balise racine de la page. lang="fr" indique la langue du contenu au navigateur et aux lecteurs d'écran.
`<meta charset="UTF-8">`	Définit l'encodage des caractères : indispensable pour que les accents et emojis/icônes s'affichent correctement.
`<meta name="viewport" ...>`	Force l'affichage à la largeur réelle de l'écran (width=device-width) sans zoom automatique (initial-scale=1.0), essentiel pour le mobile.
`<title>`	Texte affiché dans l'onglet du navigateur et dans les résultats de recherche.
`<link href="...bootstrap..." rel="stylesheet">`	Charge la feuille de style Bootstrap 5 depuis un CDN externe : fournit la grille (row / col-md-4) et des classes utilitaires.
`<link rel="stylesheet" href="css/style.css">`	Charge le CSS personnalisé du projet, en local. Placé après Bootstrap pour pouvoir le surcharger.
`<link href="...font-awesome...">`	Charge la bibliothèque d'icônes Font Awesome, utilisée par toutes les balises <i class="fas fa-...">.

1.2 Corps de page et navigation

Balise / attribut	Rôle
`<body id="page-home">`	Contient tout le contenu visible. L'id identifie la page (utile pour du CSS ciblé).
`<nav class="navbar navbar-expand-lg sticky-top">`	Balise sémantique de menu de navigation. sticky-top la fait rester collée en haut au défilement.
`<div class="container">`	Classe Bootstrap qui centre le contenu avec des marges latérales automatiques.
`<a class="navbar-brand">`	Logo/nom du site, cliquable, ramène à l'accueil.
`<i class="fas fa-graduation-cap"></i>`	Icône Font Awesome (balise <i> vide, stylée en icône par la classe CSS).
`<ul class="navbar-nav ms-auto flex-row flex-wrap">`	Liste des liens de menu. ms-auto pousse la liste à droite ; flex-row flex-wrap force les liens à rester alignés et passer à la ligne sur petit écran (remplace le menu hamburger, retiré faute de JS).
`<a class="nav-link active">`	La classe active met en évidence visuellement le lien de la page courante.

1.3 Section Hero

Balise / attribut	Rôle
`<section class="hero">`	Bloc sémantique pour le grand bandeau d'introduction en haut de l'accueil.
`<h1>`	Titre principal de la page — un seul <h1> par page, important pour le SEO et l'accessibilité.
`<p>`	Texte d'accroche sous le titre.
`<a href="courses.html"`	Lien stylé comme un bouton grâce à une classe CSS personnalisée.

Balise / attribut	Rôle
class="btn-primary-custom">	

1.4 Section Statistiques

Balise / attribut	Rôle
	Ligne de la grille Bootstrap : contient les colonnes.
	col-6 = 6/12 colonnes (2 par ligne) sur petit écran ; col-md-3 = 3/12 colonnes (4 par ligne) à partir d'un écran moyen.
.stat-number / .stat-label	Classes CSS personnalisées qui stylisent le chiffre en gros et le libellé en dessous.

1.5 Section Cours populaires

Balise / attribut	Rôle
	Conteneur en CSS Grid (défini dans style.css) qui range les cartes automatiquement en colonnes responsives.
	Carte de cours autonome.
.course-card-img	Bandeau coloré contenant l'icône du cours.
.course-card-tag	Badge de catégorie (, car texte en ligne).
<h3 class="course-card-title">	Titre du cours (h3 : on descend d'un niveau depuis le h1 de la page).
.course-card-meta	Ligne affichant la durée et la note.
<a href="course-4.html" class="btn-voir-cours">	Lien vers la page de détail statique du cours. Remplace l'ancien course.html?id=4 (routage par JavaScript).

1.6 Pied de page (footer)

Même logique de grille Bootstrap (row / col-md-4, trois colonnes égales) que la section statistiques, pour organiser logo, liens rapides et informations de contact. La classe list-unstyled retire les puces par défaut des listes.

2. courses.html — Catalogue des cours

Cette page reprend exactement le même <head>, la même navigation et le même pied de page que index.html (seul le lien « Cours » porte désormais la classe active). Seules les parties propres à cette page sont détaillées ici.

2.1 Bandeau d'en-tête

Balise / attribut	Rôle
`<section class="search-bar">`	Bandeau bleu en haut de page. Le nom de la classe est un reliquat de la version JavaScript, qui contenait un vrai champ de recherche ; ici c'est un simple bandeau de titre.
`<h2>`	Titre de section (pas de h1 ici : il reste implicite au niveau du <title>/contenu principal).

2.2 Grille des 6 cours

Point clé de l'architecture sans JavaScript : dans l'ancienne version, une seule fonction JS (createCourseCard) générait les 6 cartes automatiquement à partir d'un fichier de données. Ici, chaque carte est écrite à la main, une par une, dans le HTML.

Balise / attribut	Rôle
`<div class="courses-grid">`	CSS Grid : range automatiquement les cartes en colonnes (3-4 sur grand écran, 1 sur mobile), sans aucune media query grâce à repeat(auto-fill, minmax(300px, 1fr)).
`<div class="course-card">`	Une carte par cours, répétée 6 fois dans le fichier.
`<a href="course-1.html" class="btn-voir-cours">`	Chaque lien pointe directement vers une page HTML de détail dédiée (course-1.html à course-6.html), qui existe réellement sur le serveur. Remplace le paramètre d'URL ?id= lu auparavant par JavaScript.

Conséquence pratique : pour ajouter un 7e cours, il faut copier-coller un bloc … et créer un nouveau fichier course-7.html — il n'y a plus de génération automatique.

2.3 Pages de détail (course-1.html à course-6.html)

Les 6 pages suivent exactement la même structure ; seul le contenu (titre, tuteur, programme…) change. Voici la structure commune, illustrée avec course-1.html :

Balise / attribut	Rôle
`<div class="course-detail">`	Carte blanche encadrant tout le contenu du détail du cours.
`<div class="course-detail-header">`	Bloc d'en-tête : catégorie, titre, métadonnées (tuteur, durée, niveau, note).
`<span class="course-card-tag">`	Réutilisation du badge de catégorie déjà vu sur les cartes.
`<h1>`	Titre du cours — cette fois c'est bien le h1 principal de la page, car il n'y a pas d'autre titre plus important.
`<div class="meta">`	Ligne d'informations : tuteur, durée, niveau, note, nombre d'étudiants.
`<div class="course-detail-section">`	Bloc répété pour chaque partie : Description, Programme, Prérequis.
`<h3>`	Titre de chaque sous-section (Description, Programme, Prérequis).
`<ul> / <li>`	Liste à puces du programme du cours (syllabus).
`<a href="courses.html" class="btn-primary-custom">`	Bouton de retour vers le catalogue complet.

3. tutors.html — Page des tuteurs

Structure identique à courses.html (même logique de grille), mais avec des cartes de tuteurs plutôt que des cartes de cours.

Balise / attribut	Rôle
`<div class="tutors-grid">`	Grille CSS Grid dédiée aux tuteurs (colonnes légèrement plus étroites que courses-grid : minmax(250px, 1fr)).
`<div class="tutor-card">`	Carte d'un tuteur, répétée 6 fois.
`.tutor-avatar`	Cercle coloré contenant une icône Font Awesome représentant le tuteur (le border-radius: 50% transforme le carré en cercle).
`<h3 class="tutor-name">`	Nom du tuteur. Utilise un vrai titre de section (h3) plutôt qu'un simple , pour l'accessibilité et le SEO.
`<p class="tutor-subject">`	Matière enseignée par le tuteur.
`<p class="tutor-bio">`	Courte biographie du tuteur.
`.tutor-rating`	Note affichée avec des icônes étoile Font Awesome répétées.
`style="margin-top:..." (en ligne)`	CSS en ligne, directement dans l'attribut style : raccourci utilisé pour la ligne de statistiques (cours/étudiants) plutôt que de créer une classe CSS dédiée.

4. resources.html — Ressources téléchargeables

Les ressources sont fictives (aucun fichier réel n'est fourni avec ce projet de démonstration). Les boutons de téléchargement sont donc volontairement désactivés plutôt que de pointer vers des fichiers inexistants.

Balise / attribut	Rôle
`<div class="resource-item">`	Bloc Flexbox horizontal (display: flex dans le CSS) qui aligne icône, texte et bouton sur une même ligne.
`.resource-icon`	Icône représentant le type de fichier (document ou archive).
`.resource-info`	Bloc contenant le titre et la description de la ressource.
`.resource-title / .resource-meta`	Nom de la ressource, puis description + type + taille du fichier.
`<span class="resource-btn resource-btn-disabled">`	Bouton de téléchargement volontairement désactivé (span et non <a>) : pas de navigation possible, curseur « interdit », attribut title affichant une infobulle explicative.
`<p class="text-center text-muted mt-4">`	Classes utilitaires Bootstrap : texte centré (text-center), grisé (text-muted), marge au-dessus (mt-4).

5. contact.html — Formulaire de contact

C'est la page qui a le plus changé par rapport à la version avec JavaScript : toute la validation et l'envoi reposent désormais sur des mécanismes HTML natifs, sans aucun script.

5.1 Le formulaire et son envoi

Balise / attribut	Rôle
`<p class="form-note">`	Encadré d'avertissement informant l'utilisateur que le formulaire ouvre son client mail (pas de serveur pour traiter l'envoi).
`<form action="mailto:..." method="post" enctype="text/plain">`	Au lieu d'envoyer les données à un serveur, le navigateur ouvre le client de messagerie par défaut avec un brouillon pré-rempli.
`enctype="text/plain"`	Formate les données du formulaire en texte lisible dans le corps du mail plutôt qu'en format technique illisible.
`name="Nom" / name="Email" / ...`	Attribut essentiel ici : c'est le nom qui apparaît comme libellé dans le mail généré.

5.2 Validation native (remplace le JavaScript)

Attribut	Rôle
`required`	Le champ ne peut pas être laissé vide ; le navigateur bloque l'envoi et affiche une bulle d'erreur automatique.
`minlength="2" / minlength="10"`	Impose un nombre minimal de caractères (nom, message).
`type="email"`	Le navigateur vérifie seul le format d'une adresse e-mail (xxx@xxx.xxx).
`<select> + <option required>`	Liste déroulante ; le choix de la première option vide combiné à required empêche de valider sans sélection.

Ces attributs remplacent entièrement l'ancienne fonction JavaScript validateContactForm() : c'est le navigateur, et non un script, qui contrôle chaque champ avant de permettre l'envoi.

6. css/style.css — Feuille de style

Fichier unique partagé par toutes les pages. Il est organisé en sections commentées, résumées ci-dessous.

6.1 Variables CSS et réinitialisation

Sélecteur / propriété	Rôle
`:root { --couleur-principale: ... }`	Variables CSS personnalisées : une couleur définie une seule fois puis réutilisée partout via var(--couleur-principale). Modifier une seule ligne change toute la palette du site.
`* { margin: 0; padding: 0; box-sizing: border-box; }`	Réinitialise les marges par défaut (différentes selon les navigateurs) ; box-sizing: border-box inclut le padding et la bordure dans la largeur totale d'un élément.
`body { font-family, color, background, line-height }`	Police, couleur de texte, fond et interligne par défaut de toute la page.
`a { text-decoration: none; color: inherit; }`	Retire le soulignement automatique des liens et leur fait hériter la couleur du texte parent.

6.2 Navigation

Sélecteur	Rôle
`.navbar`	Fond blanc forcé (!important nécessaire pour surcharger le style Bootstrap par défaut), ombre légère.
`.nav-link:hover, .nav-link.active`	Sélecteur groupé (virgule = « l'un OU l'autre ») : applique le même style (texte bleu, fond bleu clair) au lien survolé et au lien actif.

6.3 Cartes de cours

Sélecteur / propriété	Rôle
`.course-card { display: flex; flex-direction: column; height: 100%; }`	Flexbox en colonne ; combiné à height: 100%, garantit que toutes les cartes d'une même rangée ont la même hauteur.
`.course-card-body { flex-grow: 1; }`	Cette zone « grandit » pour occuper l'espace restant, ce qui pousse le bouton en bas de chaque carte, aligné entre les cartes.
`.course-card:hover { transform: translateY(-5px); }`	Au survol, la carte remonte de 5 pixels ; combiné à la transition, donne un effet fluide plutôt qu'un saut brusque.

6.4 CSS Grid

Propriété	Rôle
`display: grid;`	Active la mise en page en grille sur le conteneur.
`grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));`	Crée autant de colonnes que possible, chacune large d'au moins 300px, étirées pour occuper l'espace restant. Donne un site responsive sans aucune media query.
`gap: 1.5rem;`	Espace entre les cartes, horizontal et vertical.

6.5 Formulaire de contact

Sélecteur	Rôle
`.form-group input, .form-group textarea, .form-group`	Sélecteur groupé : applique le même style de bordure et de

Sélecteur	Rôle
`select`	padding aux trois types de champs en une seule règle.
`:focus`	Style appliqué quand l'utilisateur clique dans un champ (bordure bleue, légère ombre).
`:invalid:not(:placeholder-shown)`	Cible un champ qui ne respecte pas ses règles de validation, mais seulement s'il n'est plus vide. Évite d'afficher une erreur avant même que l'utilisateur ait commencé à saisir.

6.6 Media query responsive

Règle	Rôle
`@media (max-width: 768px) { ... }`	Bloc de règles qui ne s'applique que sur un écran de 768px de large ou moins (tablette/mobile) : titres réduits, grilles forcées à une colonne, éléments de ressource empilés verticalement.

7. Synthèse

Le passage d'une architecture JavaScript à un site 100% statique a nécessité les changements suivants, résumés ici :

Avant (avec JavaScript)	Après (sans JavaScript)
`Une fonction génère les cartes de cours à partir de data.js`	Chaque carte est écrite en dur dans le HTML
`course.html?id=3 lu par JavaScript`	Une page HTML dédiée par cours (course-1.html à course-6.html)
`Recherche en temps réel via un écouteur d'événement`	Fonctionnalité retirée (nécessite du JavaScript)
`Validation du formulaire en JavaScript (validateContactForm)`	Validation native du navigateur (required, type, minlength)
`Envoi du formulaire intercepté par JavaScript (preventDefault)`	Envoi via action="mailto:..." qui ouvre le client mail
`Téléchargement simulé par une alert() JavaScript`	Bouton visuellement désactivé (), sans lien cassé
`Menu hamburger géré par le JavaScript de Bootstrap`	Liens de navigation toujours visibles, qui passent à la ligne en CSS

Ces choix respectent la contrainte du sujet (aucun JavaScript) tout en conservant une expérience utilisateur cohérente et un code lisible, organisé et commenté.